

#1:

1. What is the value returned by the following function? Express your answer as a function of n . Using big- O notation, give the worst-case running time.

```
function Algorithm1 (n);
1.  x := 0;
2.  for i := 1 to n-1 do
3.    for j := i+1 to n do
4.      for k := 1 to j do
5.        x := x + 1;
6.  return (x);
```

The Value Returned is n^3 , and the big O notation is $O(n^3)$

2. Exercise 2.3-5, page 44 of your text.
You can also think of insertion sort as a recursive algorithm. In order to sort $A[1..n]$, recursively sort the subarray $A[1..n-1]$ and then insert $A[n]$ into the sorted subarray $A[1..n-1]$. Give a recurrence for the worst-case running time of this recursive version.

Recurrence $T(n) = T(n-1) + n$

doing $T(n-1)$ recursively until $T(n)=1$, $(T(n-2) = T(n-3) + (n-2)) \dots$
then add a n of summation to $O(n^2)$

3. Write pseudocode for the above recursive version of the Insertion Sort algorithm.

if there isn't anything, return.

Else, recall insertion sort
then set x to
the current value in the array.

Another value is the previous
value in the array. Go
backwards through the array
setting the current position's
value to the next one,
then increment.

~~if (n = 0) { return 0; }
else {
 insertionSort(A, n-1);
 x = A(n);
 // ...~~

4. Write a linear-time ($\Theta(n)$) algorithm that sorts n **distinct** integers, ranging in size between 1 and kn inclusive, where k is a constant positive integer. (Hint: Use a kn -element array. However, do not write an algorithm as in Ch 8 that works even when the numbers are not all distinct since we want a simpler algorithm.) ALSO, discuss briefly why this algorithm, despite being linear, is **neither popular nor practical, nor general (e.g., might not work for non-numeric data)**.

$[0 \dots kn]$

* I had to go to Reddit to learn about Radix Sort

• From my understanding, you can sort the algorithms by going by digit place, moving higher digit numbers along based on if they still have digits, doing a $\% 10$
Ex. Array(x) ...

radix sort(Array) {

for (each element) {

for (each digit (1-9))

move in Array to beginning/end

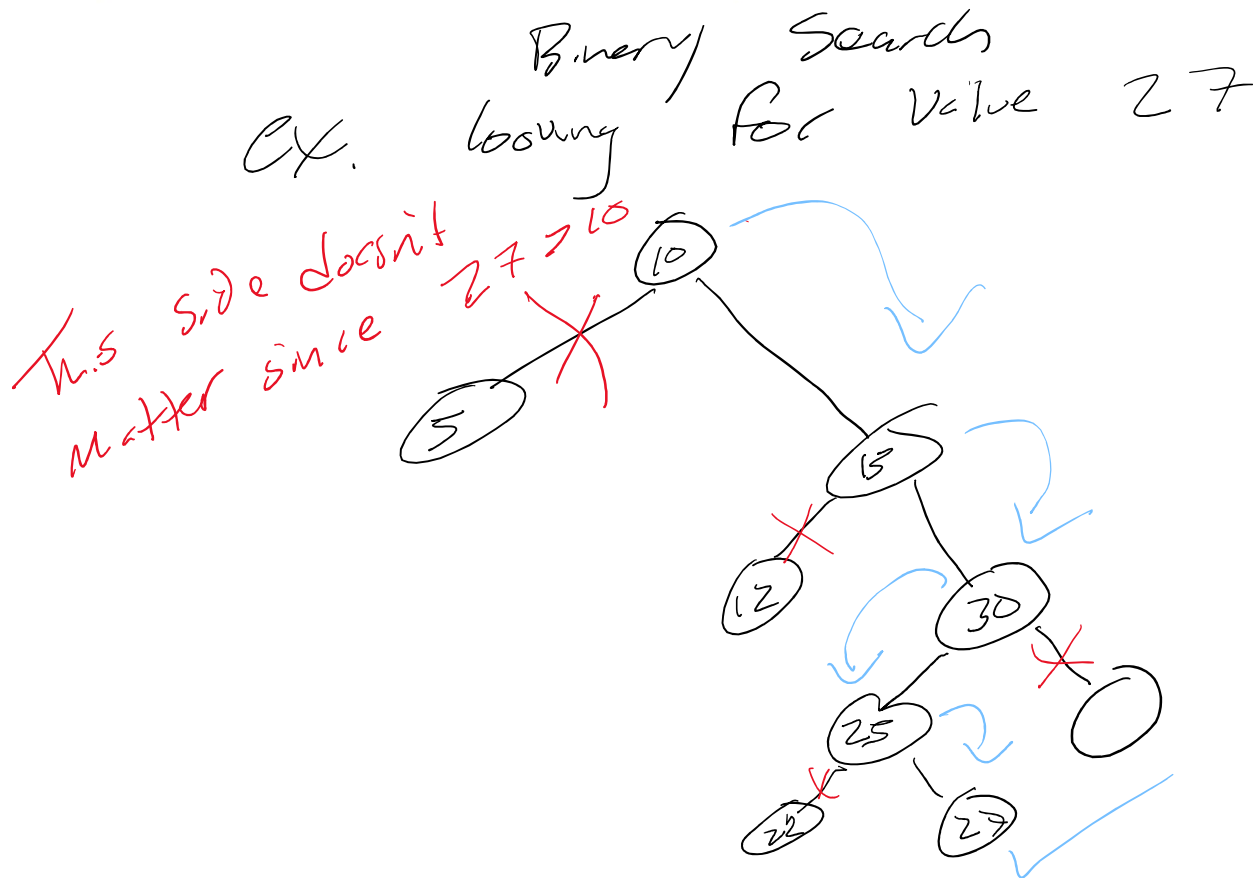
}

}

From what I learned it may not work on all
datatypes, or sizes of variable length.

5. Exercise 2.3-6, page 44-45 of your text.

Referring back to the searching problem (see Exercise 2.1-4), observe that if the subarray being searched is already sorted, the searching algorithm can check the midpoint of the subarray against v and eliminate half of the subarray from further consideration. The **binary search** algorithm repeats this procedure, halving the size of the remaining portion of the subarray each time. Write pseudocode, either iterative or recursive, for binary search. Argue that the worst-case running time of binary search is $\Theta(\lg n)$.



For each level that is iterated through,
the options for the search is split in half.
With the worst case scenario being $\log_2 n$.
It is possible that the BEST case is better,
but the worst case will continuously break the
tree into two parts, disregarding the part
that doesn't follow the check (ex. left
for $<$ or Right for $>$).

Pseudo code:

Tree

if (found) { return }

else { do recursive search,

if \$current value is $>$ than key,

If {current value is $>$ than key,
go left

else { go right }

}

Completely depends on the tree being a BST.

6. Problem 3-2, page 71 of your text (**Relative asymptotic growths**).

3-2 Relative asymptotic growths

Indicate, for each pair of expressions (A, B) in the table below, whether A is O , o , Ω , ω , or Θ of B . Assume that $k \geq 1$, $\epsilon > 0$, and $c > 1$ are constants. Your answer should be in the form of the table with "yes" or "no" written in each box.

	A	B	O	o	Ω	ω	Θ
a.	$\lg^k n$	n^ϵ	yes	yes	no	no	no
b.	n^k	c^n	yes	yes	no	no	no
c.	$\sqrt{n}$	$n^{\sin n}$	no	no	no	no	no
d.	2^n	$2^{n/2}$	no	no	yes	yes	no
e.	$n^{\lg c}$	$c^{\lg n}$	yes	no	yes	no	yes
f.	$\lg(n!)$	$\lg(n^n)$	yes	no	yes	no	yes

$\log < \text{poly}$
 $\text{poly} < \text{exp}$

$-1 \leq \sin \leq 1$

$c > \frac{1}{2}$

- lecture, these are equal

$O \leq, \Omega \geq, \Theta =, o <, \omega >$

c. $a^{\log_b c} = c^{\log_b a}$

$$a^{19} = c^{11}$$

$$\lg(n!) = \Theta(n \lg n)$$

